

Java Data Structures Mastery 

إطار عمل المجموعات في Java

دليل مهندسي البرمجيات الشامل لفهم واستخدام Java Collections Framework وتطبيقاتها العالية الأداء

لماذا نحتاج المجموعات؟

ما هو Java Collections Framework؟



بيئة متكاملة موحدة تحتوي على هياكل بيانات (Data Structures) وخوارزميات جاهزة
ومحسّنة لتخزين وإدارة البيانات بكفاءة عالية

المصفوفات (Arrays) vs المجموعات (Collections)

1. المصفوفات العادية (Arrays)

الحجم ثابت بعد الإنشاء ولا يمكن تغييره ديناميكياً.

تخزن أنواع البيانات الأولية (Primitives) وكذلك الكائنات (Objects).

تفتقر إلى خوارزميات جاهزة للفرز والبحث والتحكم المتقدم.

2. إطار المجموعات (Collections)

الحجم ديناميكي ويتمدد/ينكمش تلقائياً حسب البيانات.

تخزن الكائنات فقط (Objects) وتعتمد على Wrapper Classes لـ Primitives.

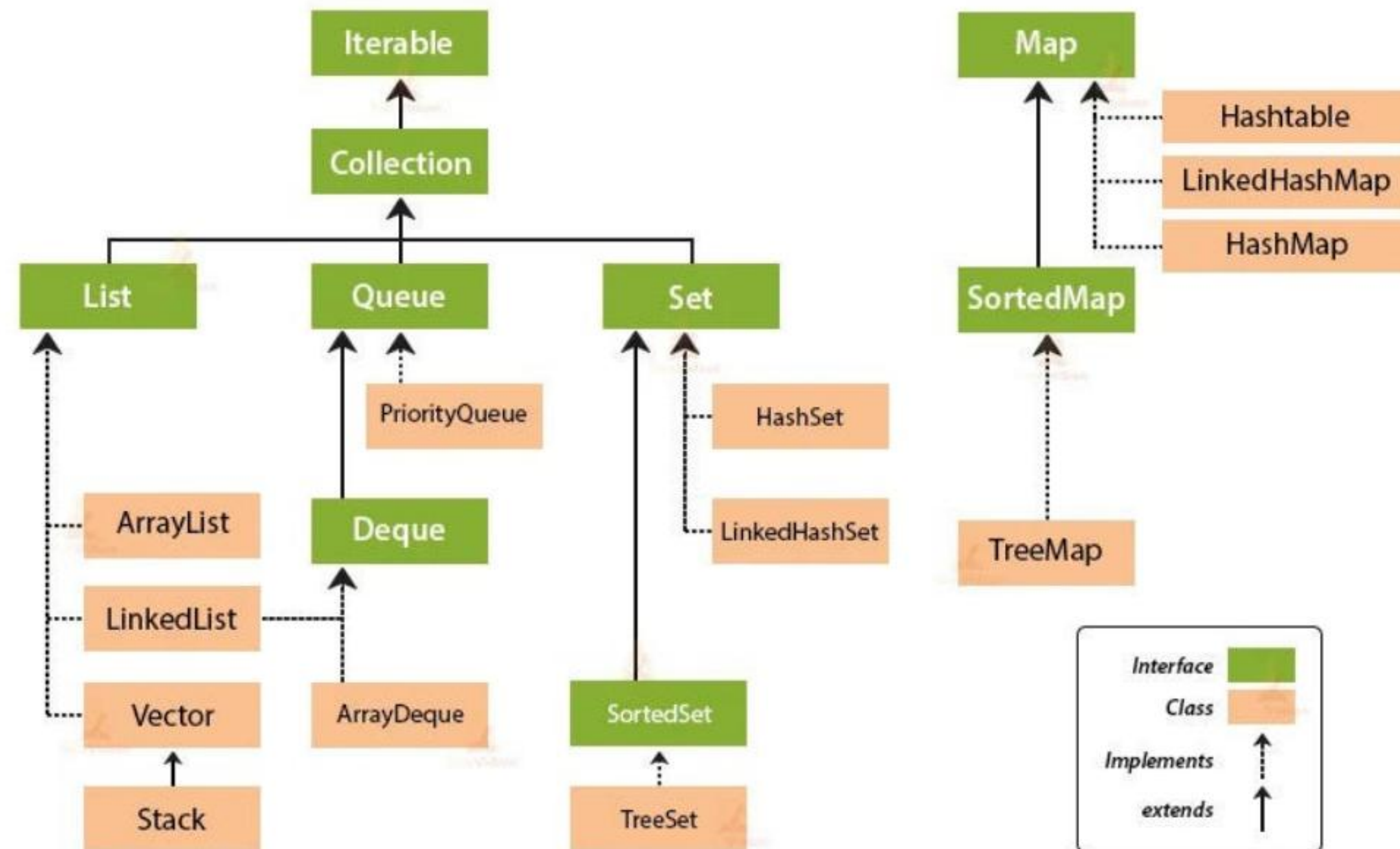
توفر خوارزميات فائقة الكفاءة جاهزة لـ Sorting و Searching و Filtering.

الهيكل الهرمي (Hierarchy of Collections)

التسلسل الهرمي للواجهات (Interfaces)

- **Iterable Interface**: الواجهة الجذرية الأب التي تسمح بالتكرار (Iteration) باستخدام for-each.
- **Collection Interface**: الواجهة الرئيسية المباشرة لـ List و Set و Queue.
- **Map Interface**: تعمل بشكل مستقل بأسلوب الأزواج (Key-Value Pairs) ولا ترث مباشرة من Collection.

Collection Framework Hierarchy in Java



الواجهات الأساسية

أبرز عائلات المجموعات في Java

List • Set • Queue • Map

واجهة القوائم: ArrayList vs LinkedList

ArrayList .1

تعتمد على مصفوفة ديناميكية. ممتازة للوصول السريع عبر المؤشر
Random Access $O(1)$ ولكنها بطيئة في الإدراج والحذف بمنتصف
القائمة $O(n)$.

```
List<String> users = new ArrayList<>(); users.add("Ahmed");  
users.add("Sara"); String first = users.get(0); //  $O(1)$   
Fast
```

LinkedList .2

تعتمد على Doubly Linked List. سريعة جداً في الإدراج والحذف $O(1)$
وبطيئة في الوصول العشوائي للآليات $O(n)$.

```
List<String> tasks = new LinkedList<>(); tasks.add("Task  
1"); tasks.addFirst("Priority Task"); //  $O(1)$ 
```


واجهة المجموعات الفريدة: HashSet vs TreeSet

1. HashSet (غير مرتبة - فائقة السرعة)

تمنع التكرار (Unique values). تعتمد على Hashing مع أداء $O(1)$ للإضافة والبحث والحذف بدون ضمان ترتيب.

```
Set<Integer> set = new HashSet<>(); set.add(10);  
set.add(10); // Ignored (No duplicates)
```

2. TreeSet (مرتبة تلقائياً)

تخزن العناصر بشكل مرتب تصاعدياً باستخدام شجرة Red-Black Tree. تعطي أداء زمني قدره $O(\log n)$.

```
Set<Integer> tree = new TreeSet<>(); tree.add(40);  
tree.add(10); // Stores automatically sorted: [10, 40]
```


واجهة الطوابير: Queue & Deque

1. PriorityQueue (طابور الأولوية)

يعالج العناصر حسب الأولوية (Natural Ordering أو Custom Comparator) بدلاً من ترتيب الدخول FIFO.

```
Queue<Integer> pq = new PriorityQueue<>(); pq.offer(30);  
pq.offer(10); int min = pq.poll(); // Returns 10 (Highest  
priority)
```

2. ArrayDeque (طابور مزدوج الإدخال)

أسرع من Stack و LinkedList عند الاستخدام ك Stack أو Queue لعدم وجود تكلفة إضافية للذاكرة Nodes.

```
Deque<String> deque = new ArrayDeque<>();  
deque.push("First"); // Stack LIFO operation deque.pop();
```


واجهة الخرائط: HashMap vs TreeMap

1. HashMap (مفتاح-قيمة)

تخزن البيانات على شكل مفاتيح وقيمة Key-Value. توفر سرعة أداء ثابتة $O(1)$ في البحث والتخزين.

```
Map<String, Double> map = new HashMap<>();  
map.put("Salary", 7500.50); double val = map.get("Salary");  
// O(1)
```

2. TreeMap (مفاتيح مرتبة)

ترتب المفاتيح طبقاً للقيم أو ال Comparator المخصص. زمن المعالجة $O(\log n)$ لعمليات البحث والإضافة.

```
Map<String, Integer> scores = new TreeMap<>();  
scores.put("Zara", 90); scores.put("Adam", 100); // Sorted  
by keys: Adam → Zara
```


مقارنة التعقيد الزمني (Time Complexity)

Deletion (الحذف)	Insertion (الإدراج)	Search (البحث)	Access (الوصول)	Data Structure
$O(n)$	$O(n)$ [$O(1)$ Amortized at end]	$O(n)$	$O(1)$	ArrayList
$O(1)$	$O(1)$	$O(n)$	$O(n)$	LinkedList
$O(1)$	$O(1)$	$O(1)$	N/A	HashSet / HashMap
$O(\log n)$	$O(\log n)$	$O(\log n)$	N/A	TreeSet / TreeMap

قواعد التحسين والأداء في الإنتاج

أفضل الممارسات (Best Practices)

تحديد السعة المبدئية Initial Capacity لـ ArrayList و HashMap لتجنب إعادة التكبير المكلفة (Re-hashing).

استخدام Set دائماً بدلاً من القوائم عند التحقق من وجود عناصر لمنع التكرار والحصول على أداء $O(1)$.

استخدام ConcurrentHashMap في البيئات متعددة الخيوط (Multi-threading) بدلاً من Hashtable القديمة.

$O(1)$

معدل الأداء المستهدف للأنظمة الضخمة